

Part 1: The Phantom Bug

- Write a program that tosses \$50,000,000\$ random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

```
PS C:\Users\mbekn\Desktop> python part1.py
Run 1: Pi = 3.14168 (Total Hits: 39271026)
Run 2: Pi = 3.14128 (Total Hits: 39265985)
Run 3: Pi = 3.14107 (Total Hits: 39263352)
Run 4: Pi = 3.14137 (Total Hits: 39267159)
Run 5: Pi = 3.14116 (Total Hits: 39264509)
```

Part 2: The Synchronization Trap

- Fix the race condition by adding synchronized to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

```
PS C:\Users\mbekn\Desktop> python part2.py
Single-threaded: Time = 10075.27 ms, Pi = 3.14147
Multi-threaded (Locked): Time = 18003.17 ms, Pi = 3.14126
```

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with \$100,000,000\$ iterations.

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T/N T1)	Efficiency (T/Speedup)
1	20835.92	1.0x	100%
2	10782.97	1.93x	96.6%
4	6382.83	3.26x	81.6%
8	4598.83	4.53x	56.6%
16	4413.63	4.72x	29.5%

Questions

1. *Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?*

There are only 8 physical cores, so hardware can only run 8 threads concurrently. Running 16 threads creates resource contention and context-switching overhead rather than doubling performance.

2. *Why was the synchronized version in Part 2 slower than running on one single core?*

Severe lock contention causes cores to spend ~95% of their cycles stalling and synchronizing cache lines. The heavy overhead of locking makes it much slower than single-threaded sequential execution without locks.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).